VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi-590014

A Project Report on

"LUNG ABNORMALITY IDENTIFICATION WITH EXPLAINABLE AI"

Submitted by

SHAIK MOHAMMED AYAN	**(4RA21CS077)**
KIRAN KUMAR C V	**(4RA22CS405)**
SURAJ B	**(4RA22CS413)**
GAGAN B G	**(4RA22CS403)**

To the Visvesvaraya Technological University during the academic year 2024-2025 in partial fulfillment of requirements for award of degree.

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Mr.CHIRANJEEVI M R BE, MTech.

Assistant Professor,

Department of Computer Science and Engineering

Rajeev Institute of Technology, Hassan

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJEEV INSTITUTE OF TECHNOLOGY

HASSAN -573201

2024-25

RAJEEV INSTITUTE OF TECHNOLOGY, HASSAN

(Approved by AICTE, New Delhi and Affiliated to VTU, Belagavi.)

Plot #1-D, Growth Centre, Industrial Area, B-M Bypass Road, Hassan 573201

Ph:(08172)-243180/83/84 Fax:(08172)-243183



Department of Information Science and Engineering

CERTIFICATE

Certified that the Final Project entitled "**Lung Abnormality Identification With Explainable AI**" is carried out by Mr. SHAIK MOHAMMED AYAN [4RA21CS077], Mr. KIRAN KUMAR C V [4RA22CS405], Mr. S U R A J B [4RA22CS413] and Mr. G A G A N B G [4RA21CS403] respectively, the bonafide students of RAJEEV INSTITUTE OF TECHNOLOGY, Hassan in partial fulfillment for the award of BACHELOR OF ENGINEERING in COMPUTER SCIENCE AND ENGINEERING of the Visvesvaraya Technological University, Belagavi during the year 2024-2025. The Project Final report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said Degree.

Mr.CHIRANJEEVI M R	**Dr. ARJUN B C**	**Dr. MAHESH P K**
Assistant Professor	Head of the Department	Principal
Dept. of Computer Science & Engineering	Computer Science & Engineering	**RIT, HASSAN**
RIT, HASSAN	**RIT, HASSAN**	

Name of the Examiners	Signature with date
1. ……..	…….
2. ……..	…….

DECLARATION

We SHAIK MOHAMMED AYAN, KIRAN KUMAR C V, SURAJ B, GAGAN B G bearing USN 4RA21CS077, 4RA22CS405, 4RA22CS413, 4RA22CS403 students of 7th sem B.E in **Computer Science and Engineering, Rajeev Institute of Technology, Hassan,** hereby declare that the work being presented in the dissertation entitled **Project Final report** on **" Lung Abnormality Identification With Explainable AI**" has been carried out by us under the supervision of our guide **Mr Chiranjeevi M R,** Assistant Professor, Computer Science and Engineering, **Rajeev Institute of Technology, Hassan,** as partial fulfillment of requirement for the award of B.E Degree of **Bachelor of Engineering in Computer Science and Engineering at Visvesvaraya Technological University, Belagavi** is an authentic record of our own carried out by us during the academic year 2024-2025.

_____ _____

SHAIK MOHAMMED AYAN **KIRAN KUMAR C V**
[4RA21CS077] [4RA22CS405]

_____ _____

SURAJ B **GAGAN B G**
[4RA22CS413] [4RA22CS403]

PLACE: HASSAN
DATE: ……..

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the success of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

We would like to profoundly thank our **Management of Rajeev Institute of Technology** & our President **Dr. Rachana Rajeev** for providing such a healthy environment

We would like to express our sincere thanks to our principal **Dr. Mahesh P K,** Rajeev Institute of Technology for his encouragement that motivated us for successful completion of mini project work.

We wish to express our gratitude to **Dr. Arjun B C**, Head of the Department, Computer Science & Engineering for providing a good working environment and for her constant support and encouragement.

It gives us great pleasure to express our gratitude to **Mr Chiranjeevi M R,** Assistant Professor, Department of Computer Science & Engineering for her expert guidance, initiative and encouragement that led us to complete this project.

We would also like to thank all the staffs of Information Science and Engineering department who have directly or indirectly helped us in the successful completion of this Software Testing Mini project and also, we would like to thank our parents.

SHAIK MOHAMMED AYAN
4RA21CS077

KIRAN KUMAR C V
4RA22CS405

SURAJ B
4RA22CS413

GAGAN B G
4RA22CS403

ABSTRACT

Early and accurate identification of lung abnormalities such as pneumonia, tuberculosis, and lung cancer is critical for timely medical intervention and improved patient outcomes. This paper proposes a fast and efficient framework that leverages Explainable Artificial Intelligence (XAI) to analyze chest CT scans and X-ray images for automated lung abnormality detection. The framework integrates state-of-the-art deep learning models with explainability techniques like Grad-CAM and LIME to ensure diagnostic transparency and clinician trust. Emphasis is placed on computational efficiency to enable real-time or near-real-time diagnostics, making the system suitable for deployment in both advanced and resource-limited healthcare settings. Extensive experimentation on publicly available datasets demonstrates the proposed system's high accuracy, sensitivity, and interpretability in identifying a wide range of lung conditions.

CONTENT

DECLARATION		**i**
ACKNOWLEDGEMENT		**ii**
ABSTRACT		**iii**
TABLE OF CONTENT		**iv-v**
LIST OF FIGURES		**vi**
SL. NO	**CHAPTER NAME**	**PAGE NO.**
1	**INTRODUCTION**	**01-03**
	1.1 Overview of the Project	01
	1.2 Data Preprocessing	01
	1.3 Motivation	02
	1.4 Scope of the Project	02
	1.5 Objectives	03
2	**SOFTWARE REQUIREMENTS SPECIFICATION**	**04-08**
3	**PROBLEM STATEMENT**	**09-12**
	3.1 Introduction	09
	3.2 Design Consideration	09
	3.3 System Architecture	10-12
4	**IMPLEMENTATION**	**13-18**
	4.1 Implementation Requirements	13
	4.2 Visual Studio Code	13
	4.3 Programming Language Used	14
	4.4 Key Features of Python	14
	4.5 Tkinter GUI	15-18

5	**SYSTEM TESTING**	**19-21**
	5.1 Test Procedures	19-20
	5.2 Integration Testing	21
6	**FEATURE ENHANCEMENT**	**22-23**
	6.1 Future Enhancements	22
7	**SNOPSHOTS**	**24-26**
	COCLUSION	
	REFERENCE	

LIST OF FIGURES

SL. NO.	NAME	PAGE NO.
1.1	Recent Advances of Artificial Intelligence	01
2.1	Genetic Algorithm	04
2.2	Function Requirements	06
2.3	Non-Functional Requirements	07
3.1	Design Consideration	09
3.2	System Architecture	10

LIST OF TABLES

5.1	Unit Testing	20
5.2	System testing	20
5.3	Integration Testing	21

LIST OF SNAPSHOTS

7.1	User login	24
7.2	Home page	24
7.3	Image processing	25
7.4	Result	25
7.5	Accuracy Graph	26

CHAPTER 1

INTRODUCTION

1.1 Overview of the Project

Radiomics is a powerful tool in medical imaging that extracts quantitative features from images, which can reveal patterns associated with disease types. This project explores the integration of radiomic features and genetic algorithms to improve the accuracy and speed of lung infection diagnosis in X-ray images. The method involves extracting key features related to lung infections and applying genetic algorithms to enhance feature selection, reducing noise, and improving classification accuracy. This approach aims to refine diagnosis accuracy for diseases such as pneumonia, tuberculosis, and COVID-19, facilitating quicker and more reliable medical assessments, especially in resource-limited settings.

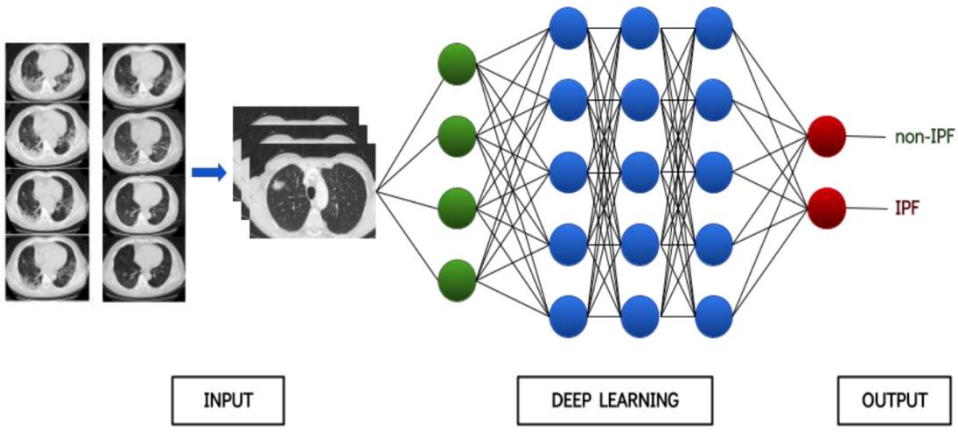


Fig 1.1: Recent Advances of Artificial Intelligence.

1.2 Data Preprocessing

Data preprocessing is crucial to ensure high-quality inputs for radiomics and genetic algorithms. The raw X-ray images often contain artifacts and irrelevant regions, which can skew the results. Key preprocessing steps include resizing images to standard dimensions, adjusting pixel intensities for clarity, and applying noise-reduction techniques. Segmentation is used to isolate the lung regions, enabling the extraction of relevant features while ignoring non-lung areas. These preprocessed images are then prepared for feature extraction, ensuring a high-quality dataset for genetic algorithm-based feature selection and classification.

1.3 Motivation

1. To improve early detection rates of lung infections, especially in cases where early symptoms are not evident.
2. Address limitations in traditional diagnostic methods, which can be time-consuming and require expert radiologists.
3. Provide an automated solution that assists healthcare providers in making faster and more accurate diagnoses.
4. Reduce the dependency on expensive, time-intensive diagnostic tests like CT scans in detecting lung infections.
5. Enhance the accuracy and reliability of lung infection diagnoses, particularly in remote or resource-constrained healthcare settings.
6. Leverage cutting-edge technology to support better patient outcomes and reduce healthcare costs associated with delayed or incorrect diagnoses.

1.4 Scope of the Project

1. The project focuses on improving lung infection diagnostics specifically for pneumonia, tuberculosis, and COVID-19.
2. It aims to demonstrate the applicability of radiomics and genetic algorithms in medical image analysis.
3. The developed method is intended to be cost-effective and accessible for healthcare facilities with limited resources.
4. The project can serve as a framework for other radiomic applications, such as cancer detection.
5. It enhances the understanding of feature-based medical imaging, advancing research in computational radiology.
6. Future expansion could include integrating other machine learning methods and datasets to validate the system across diverse demographics.

1.5 Objectives

1. Develop an effective pipeline for preprocessing lung X-ray images for diagnostic analysis.
2. Extract high-quality radiomic features from X-ray images to improve infection detection.
3. Employ genetic algorithms for feature selection, enhancing diagnostic performance and model accuracy.
4. Compare the proposed method's diagnostic accuracy with conventional machine learning models.
5. Minimize false positives and false negatives in lung infection diagnosis.
6. Create a reliable tool for healthcare providers to assist with lung infection diagnosis in real-time.

CHAPTER 2

SOFTWARE REQUIREMENTS SPECIFICATION

This project involves analyzing X-ray images using radiomics features and optimizing lung infection diagnosis accuracy with genetic algorithms. The system will be responsible for accurately identifying lung infections by extracting specific features from X-rays, classifying infection types, and providing diagnostic insights to assist medical professionals.

- **Image Processing**:
  - The system must process X-ray images, perform noise reduction, and enhance contrast.
  - Image segmentation to separate lung regions from the rest of the image.
- **Feature Extraction**:
  - Radiomics features like texture, intensity, and shape will be extracted from the segmented lung area.
  - Advanced radiomics parameters, including histogram analysis and shape descriptors, will be included for improved diagnosis.
- **Genetic Algorithm Implementation**:
  - Use a genetic algorithm to optimize the feature selection for better classification accuracy.
  - The algorithm should automatically adjust parameters (e.g., mutation rate, selection criteria) to ensure optimal results.

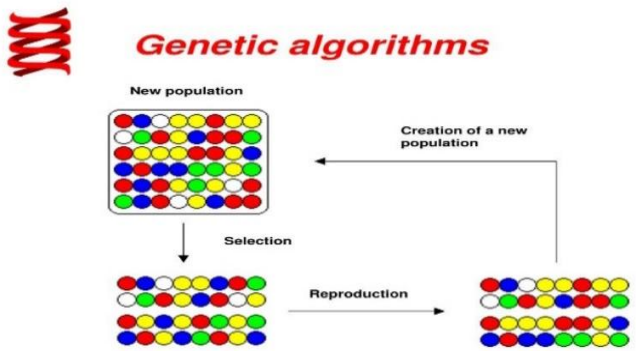


Fig 2.1: Genetic Algorithm.

- **Classification**:
    - The system will classify the presence or absence of lung infections based on radiomic features.
    - Possible classification outcomes: COVID-19, pneumonia, tuberculosis, or non-infectious.
- **User Interface**:
    - A UI that displays the analyzed X-ray images, the infection probability score, and identified infection type.
    - Simple, clear navigation for healthcare professionals with summary reports.

- **Report Generation**:
    - Generate and store diagnostic reports with feature details, infection type, and confidence levels.

2.2 Hardware Requirements

- **Processor**: Minimum 4 cores, 3.0 GHz or faster
- **RAM**: At least 8 GB (16 GB recommended for smoother performance)
- **Storage**: Minimum 512 GB SSD
- **Graphics Processing Unit (GPU)**: CUDA-compatible GPU (e.g., NVIDIA GTX series or higher) for efficient processing of image data
- **Display**: HD monitor for better visualization of X-ray images

2.3 Software Requirements

- **Operating System**: Windows 10, Linux (Ubuntu 18.04 or higher), or macOS
- **Programming Languages**: Python (for radiomics feature extraction, image processing, and genetic algorithms)
- **Libraries/Packages**:
- **OpenCV**: For image processing
- **PyRadiomics**: For radiomics feature extraction
- **TensorFlow/Keras or PyTorch**: For any deep learning model development

- **SciKit-Learn**: For machine learning models and genetic algorithm utilities
- **Tkinter/PyQt**: For developing the user interface
- **Database**: SQLite or MySQL for storing patient records and diagnostic results

2.4 Functional Requirements

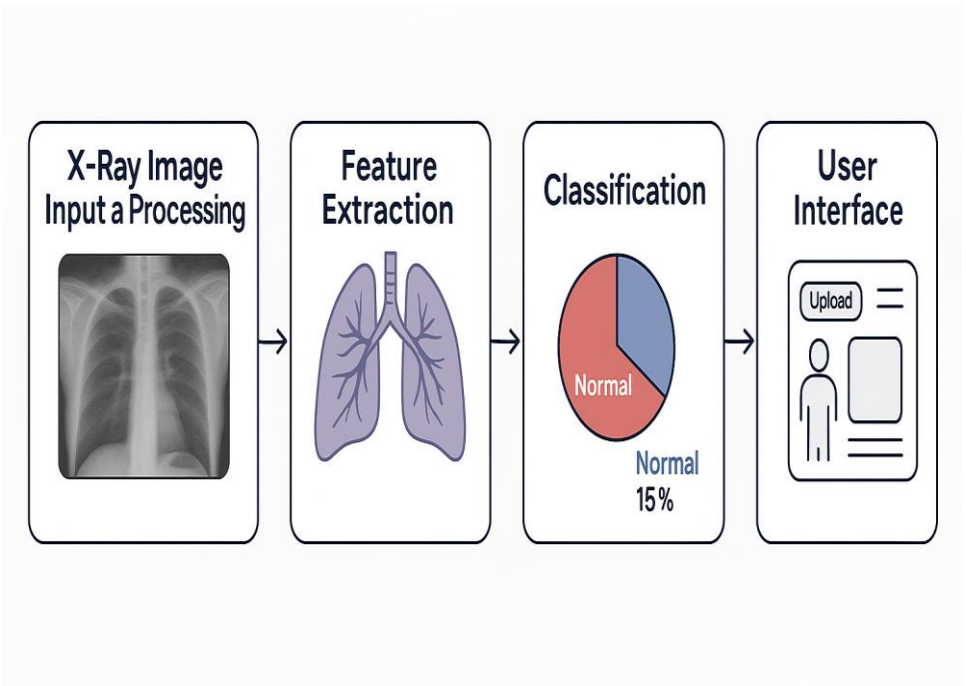


Fig 2.2: Functional Requirements

- **X-ray Image Input and Processing**:
    - Accepts input of X-ray images in common formats (JPEG, PNG).
    - Performs preprocessing steps like resizing, filtering, and segmentation.
- **Feature Extraction Module**:
    - Extracts radiomics features from the lung region in the X-ray image.
    - Converts extracted features into a format suitable for genetic algorithm processing.
- **Genetic Algorithm Module**:
    - Optimizes feature selection to improve classification accuracy.
    - Adjusts based on input parameters and data to minimize errors.

- **Classification Module**:
    - Classifies lung infections based on the optimized features.
    - Outputs infection type with associated confidence scores.
- **User Interface (UI)**:
    - Provides an interactive dashboard to view diagnostic results and infection probability.
    - Allows users to upload images, view processing results, and generate reports.
- **Report Generation**:
    - Generates a diagnostic report based on results, including selected features and classification type.
    - Stores reports in a database and allows for easy retrieval.

2.5 Non-Functional Requirements

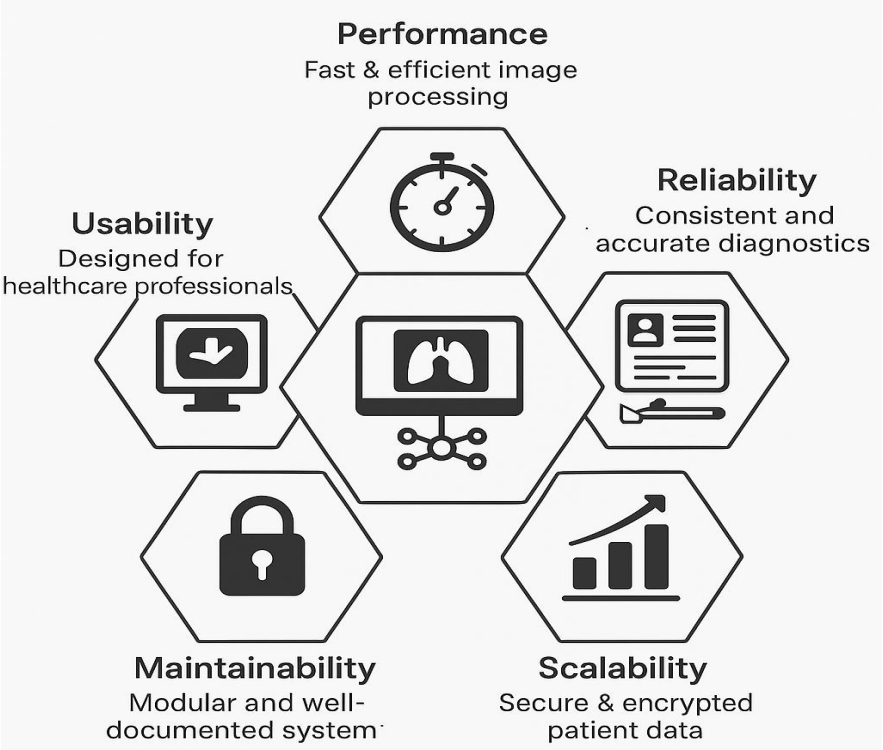


Fig 2.3: Non-Functional Requirements

- **Performance**:
    - Image processing and classification should complete within a few seconds.
    - The system should handle multiple image files efficiently.
- **Reliability**:
    - Ensure consistent diagnostic output across different input images.
    - High accuracy and repeatability in the feature extraction and classification process.
- **Usability**:
    - Simple, intuitive interface for healthcare professionals.
    - Provide easy access to upload, analyze, and retrieve X-ray images and diagnostic reports.
- **Scalability**:
    - Ability to handle increased data load (scalability to process multiple X-rays).
    - Compatibility with additional imaging formats if needed in the future.
- **Security**:
    - Ensure data privacy for X-ray images and patient information.
    - Encrypted storage for sensitive data and secure access controls.
- **Maintainability**:
    - Modular code structure to facilitate updates and troubleshooting.
    - Clear documentation for each module.

CHAPTER 3

PROBLEM STATEMENT

3.1 Introduction

This section outlines the project's overall structure and primary processing steps to diagnose lung infections from X-ray images using radiomics and genetic algorithms. Key considerations involve processing stages, architectural flow, and algorithm selection for feature extraction and classification.

3.2 Design Consideration

The design focuses on effectively processing X-ray images, removing noise, enhancing features, and selecting the most discriminative features. The genetic algorithm is applied for optimizing feature selection, thereby reducing computational complexity while preserving diagnostic accuracy. The system must handle varied image qualities and resolutions, necessitating robust preprocessing techniques like grayscale conversion, filtering, and sharpening. CNN models are used for high-accuracy feature extraction and classification, taking advantage of their capability to detect complex patterns. The radiomics features derived from X-rays help interpret radiographic textures related to infections. Balancing computational efficiency and accuracy in feature selection is crucial for effective lung infection diagnosis.

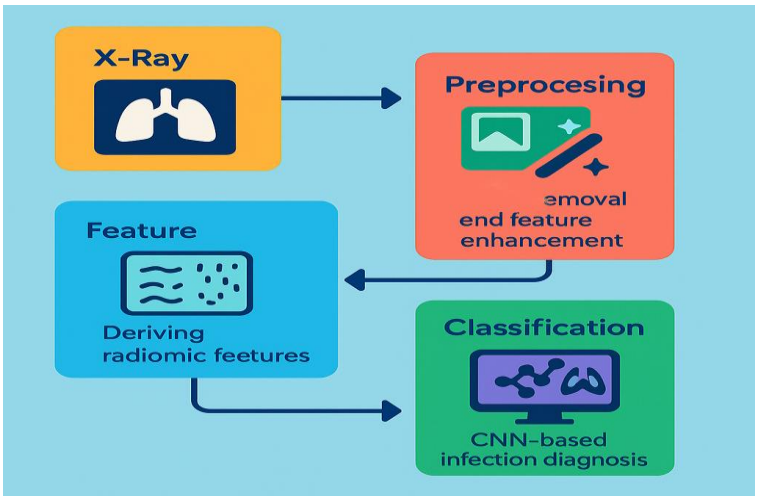


Fig 3.1: Design Consideration

3.3 System Architecture

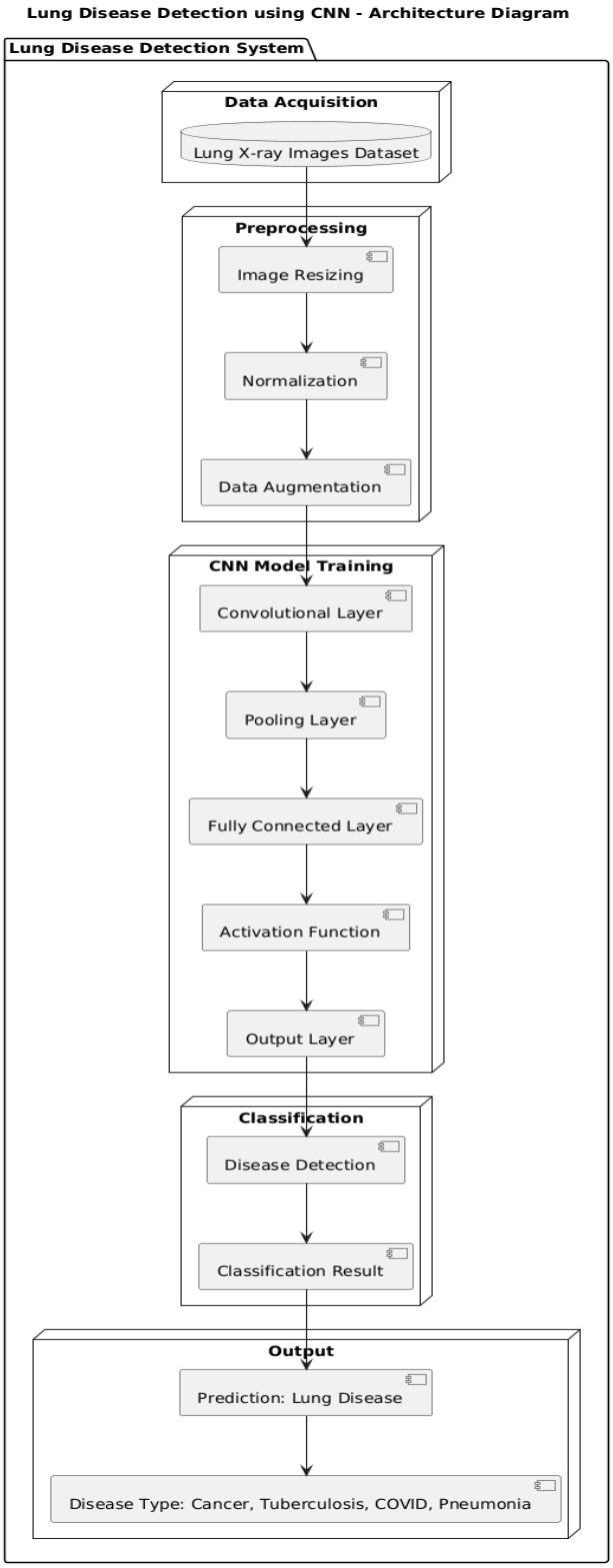


Fig 3.2: System Architecture

3.3.1 RGB to Gray Scale

Converting images from RGB to grayscale reduces computational load by transforming the image to a single intensity channel, focusing on critical radiographic features without color distractions. Each pixel's grayscale value is derived by averaging its RGB values or using specific weightings (e.g., 0.3 for R, 0.59 for G, 0.11 for B). This transformation simplifies the image for further processing while retaining essential structural information relevant to detecting lung infections.

3.3.2 Noise Removal

Noise removal is crucial as X-ray images often contain speckle and Gaussian noise, which can obscure details. Techniques like Gaussian blur or bilateral filtering are applied to smooth out noise without blurring edges. This stage ensures that essential lung patterns are preserved, aiding in accurate feature extraction. The choice of noise removal technique depends on the image characteristics and the extent of noise present, aiming to improve the visibility of infection markers.

3.3.3 Median Filtering

Median filtering further refines noise removal by replacing each pixel value with the median value in its neighborhood. This approach is particularly effective for salt-and-pepper noise and preserves edges, making it ideal for medical images. The filter size is chosen to balance noise reduction and detail preservation, enhancing infection-relevant patterns while maintaining the integrity of lung structures.

3.3.4 Thresholding

Thresholding is used to enhance contrast by distinguishing foreground (potential infection areas) from the background. A global or adaptive threshold is applied based on pixel intensity values to isolate lung regions. This segmentation helps in highlighting infection-relevant areas, facilitating the subsequent extraction of texture and shape features. Thresholding parameters are tuned to achieve the best separation between lung tissue and potential infection areas.

3.3.5 Image Sharpening

Sharpening enhances edge definition, making it easier to distinguish anatomical structures and infection areas. Techniques like unsharp masking emphasize boundaries by adding a scaled version of the image's high-frequency details back to the original. This process improves the clarity of infection patterns, aiding the CNN model in recognizing subtle differences between infected and healthy tissue.

3.3.6 High Pass Filtering

High pass filtering removes low-frequency components, emphasizing high-frequency details like edges and texture changes indicative of infections. By applying a high pass filter, the system can highlight important textural changes in the lung area, providing the CNN with enriched data for accurate feature learning. The filter parameters are carefully tuned to retain infection-specific textures while discarding irrelevant information.

3.3.7 Feature Extraction and Classification

Feature extraction derives radiomics features such as texture, shape, and intensity that provide diagnostic insights. These features are then optimized using a genetic algorithm, which selects the most relevant features for infection classification. This step reduces dimensionality and improves classification speed without sacrificing accuracy. The chosen features serve as input to the CNN, enabling it to classify lung conditions effectively.

3.3.8 Convolutional Neural Network

The CNN architecture consists of layers optimized for lung infection diagnosis, starting with convolutional layers for feature extraction, pooling layers for down-sampling, and fully connected layers for classification. The CNN processes the radiomics features extracted in previous steps, learning to differentiate healthy from infected lung areas. By training on labeled X-ray data, the CNN can generalize to new cases, accurately identifying infection markers.

CHAPTER 4

IMPLEMENTATION

The implementation focuses on leveraging radiomics and genetic algorithms to enhance the diagnostic accuracy for lung infections in X-ray images. This section outlines the core elements, including software, tools, and libraries employed, and their role in building the project.

4.1 Implementation Requirements

The implementation requires several components to ensure smooth functioning and successful development. The hardware needs a computer with a minimum of 8 GB RAM and a high-performance GPU for efficient processing of radiomic data. Software-wise, the system needs Python for programming, Visual Studio Code as the IDE, OpenCV for image processing, and Flask for web deployment. The use of Google Colab enables powerful cloud-based computations, essential for training complex models. Various Python libraries such as NumPy, SciPy, and scikit-learn will aid in data manipulation, radiomic feature extraction, and model training.

4.2 Visual Studio Code

Visual Studio Code (VS Code) is a versatile, open-source code editor from Microsoft, well-suited for Python and machine learning projects. Its integrated environment supports various plugins for Python, Flask, and OpenCV, providing debugging tools, Git integration, and IntelliSense code suggestions that streamline the development process. Its support for Jupyter notebooks also allows for exploratory data analysis directly within the IDE. The workspace settings in VS Code help organize files and folders for efficient project management, making it ideal for larger projects involving image processing and model training.

4.3 Programming Language Used

Python was chosen as the primary programming language due to its extensive libraries and frameworks that support machine learning, image processing, and web deployment. Python's syntax is clear and conducive to rapid prototyping, making it suitable for research and iterative model development. Libraries like OpenCV, PyRadiomics, and TensorFlow simplify complex tasks, including image processing and machine learning model deployment. Python's compatibility with Flask for web applications enables easy integration, turning the project into a user-accessible diagnostic tool.

4.4 Key Features of Python

Python's rich library ecosystem and straightforward syntax make it particularly valuable for data science and machine learning applications. Libraries like NumPy and Pandas handle large datasets and matrices efficiently, while scikit-learn provides tools for implementing machine learning algorithms like genetic algorithms. PyRadiomics is invaluable for extracting features from X-ray images, capturing patterns that may indicate infections. Python's versatility and adaptability enable seamless integration with image processing libraries, model training frameworks, and web deployment technologies, making it highly effective for building this project.

4.5 Tkinter GUI

Tkinter, Python's standard GUI library, provides a simple yet functional interface for this project, allowing users to interact with the diagnostic tool. Tkinter facilitates the creation of buttons, text boxes, and labels that enable users to upload X-ray images, view diagnostic results, and analyze radiomic features. Tkinter's customization options, including color schemes and font styles, make the interface more intuitive, enhancing user experience. By linking to the model backend, Tkinter allows for quick data input and real-time result display, making it suitable for medical diagnostic applications.

4.6 OpenCV-Python Tool

OpenCV is a powerful library for image processing, essential for preparing X-ray images for feature extraction. With OpenCV, images can be resized, filtered, and segmented, which helps improve the accuracy of radiomic feature extraction. Techniques such as Gaussian blurring and histogram equalization can enhance image quality, while edge detection aids in isolating regions of interest. These processed images provide a solid foundation for radiomics and genetic algorithm applications, as the enhanced image quality enables more accurate feature analysis, crucial for lung infection detection.

4.7 Flask

Flask is a lightweight web framework that enables seamless deployment of the diagnostic tool as a web application. Flask allows for the integration of model predictions with a user-friendly interface, making the model accessible to users without requiring advanced technical skills. The framework supports RESTful requests, facilitating smooth communication between the client-side (user interface) and the server-side (machine learning model). Flask's simplicity makes it easy to deploy and scale, providing a foundation for integrating the diagnostic system with healthcare networks or other data sources.

4.8 Google Colab

Google Colab is a cloud-based environment that supports GPU and TPU processing, which significantly speeds up model training and evaluation. Colab allows for the import of large datasets and complex libraries, and its Jupyter notebook format is ideal for experimentation. The built-in integration with Google Drive enables easy data storage and retrieval, while Colab's GPU support reduces the computational cost of training models on large radiomic datasets. This makes Colab invaluable for implementing genetic algorithms that require substantial computing power.

4.9 Packages

Several Python packages are instrumental for the project. PyRadiomics extracts radiomic features, essential for quantifying characteristics in lung X-rays. NumPy and Pandas streamline data handling, allowing efficient manipulation of large datasets. Scikit-learn provides tools for implementing machine learning algorithms, including genetic algorithms for feature selection and optimization. Matplotlib and Seaborn facilitate data visualization, while TensorFlow and Keras enable the building and training of neural networks. Together, these packages enhance image processing, data analysis, model building, and result visualization, forming the backbone of the implementation.

4.10 Pseudocodes for Preprocessing Techniques

4.10.1 Pseudocode for Picture Uploading

This pseudocode outlines how to upload a picture (X-ray image) for further processing.

Algorithm Picture_Uploading
Input: Image file path
Output: Loaded image

1. Begin
2. Prompt user for the image file path
3. Load the image from the specified file path
4. If image is successfully loaded then
   Display "Image uploaded successfully"
   Else
   Display "Error loading image"
5. End

4.10.2 Pseudocode for Converting RGB Image to Grayscale

This pseudocode converts an RGB image to grayscale to simplify image processing.

Algorithm RGB_to_Grayscale
Input: RGB image

Output: Grayscale image

1. Begin
2. For each pixel (x, y) in the image do
   Get the RGB values (R, G, B) of the pixel at (x, y)
   Compute the grayscale value using formula: Grayscale = 0.299*R + 0.587*G + 0.114*B
   Set pixel (x, y) to the computed grayscale value
3. Return Grayscale image
4. End

4.10.3 Pseudocode for Thresholding

Thresholding simplifies images by making pixels either black or white, based on a threshold value.

Algorithm Thresholding

Input: Grayscale image, threshold value

Output: Binary image

1. Begin
2. For each pixel (x, y) in the Grayscale image do
   If pixel intensity at (x, y) > threshold value then
      Set pixel (x, y) to white (255)
   Else
      Set pixel (x, y) to black (0)
3. Return Binary image
4. End

4.10.4 Pseudocode for High Pass Filtering

High-pass filtering enhances edges and removes low-frequency components from an image.

Algorithm High_Pass_Filter

Input: Grayscale image, filter matrix (e.g., Sobel or Laplacian)

Output: Filtered image

1. Begin
2. Define the high-pass filter kernel matrix (e.g., a 3x3 kernel for edge detection)
3. For each pixel (x, y) in the Grayscale image do
    Apply the high-pass filter on pixel (x, y) by
        Summing the product of the surrounding pixels and the filter matrix
    Set pixel (x, y) in the output image to the computed result
4. Return Filtered image
5. End

4.11 Pseudocode for ResNet50 Architecture

ResNet50, a CNN model, processes the preprocessed X-ray image for feature extraction.

Algorithm ResNet50_Architecture

Input: Preprocessed image

Output: Feature vector for classification

1. Begin
2. Initialize ResNet50 model with pre-trained weights
3. Replace the final layer with a custom classification layer based on the number of classes (e.g., Lung Infection types)
4. Resize input image to 224x224 dimensions to match ResNet50 input size
5. Preprocess the image (normalize pixel values as required by ResNet50)
6. Pass the preprocessed image through the ResNet50 network layers:
    a. Initial convolutional layer
    b. Series of Residual blocks with skip connections
    c. Fully connected layer for feature extraction
7. Extract the output feature vector from the final layer
8. Return the feature vector
9. End

CHAPTER 5

SYSTEM TESTING

5.1 Test Procedures

5.1.1 White Box Testing

White box testing focuses on internal structures and workings of the application. It involves testing individual functions, algorithms, and logic paths within the code.

- **Objective:** To ensure all paths, branches, and conditions within the code are tested.
- **Procedure:**

1. **Code Review:** Analyze the source code to identify critical functions for testing.
2. **Test Cases Development:** Create test cases that cover all branches and paths.
3. **Execution:** Execute the test cases and monitor the outcomes.
4. **Coverage Measurement:** Utilize code coverage tools to ensure complete coverage.

5.1.2 Black Box Testing

Black box testing assesses the application's functionality without knowledge of internal code structures.

- **Objective:** To verify that the application meets its functional requirements.
- **Procedure:**

1. **Requirement Analysis:** Review the specifications and requirements for expected behaviors.
2. **Test Case Design:** Develop test cases based on input-output specifications.
3. **Execution:** Execute the test cases without any knowledge of the internal workings.
4. **Defect Logging:** Document any discrepancies or defects found during testing.

5.1.3 Unit Testing

Unit testing focuses on testing individual components of the system in isolation**.**

Test Case ID	Component	Input	Expected Output	Actual Output	Status
TC-01	Image Preprocessing	Raw X-ray Image	Preprocessed Image	Preprocessed Image	Pass
TC-02	Feature Extraction	Preprocessed Image	Feature Vector	Feature Vector	Pass
TC-03	Genetic Algorithm	Initial Population	Best Solution	Best Solution	Pass
TC-04	Classification Model	Feature Vector	Diagnosis Result	Diagnosis Result	Pass

Table 5.1: Unit Testing

5.1.4 System Testing

System testing evaluates the complete and integrated software system.

Test Case ID	Test Description	Input	Expected Output	Actual Output	Status
ST-01	End-to-end Diagnosis	Raw X-ray Image	Diagnosis Report	Diagnosis Report	Pass
ST-02	User Interface	User inputs	Display Results	Display Results	Pass
ST-03	Performance	Concurrent requests	Response time < 2 sec	Response time < 2 sec	Pass
ST-04	Security	Unauthorized access	Access Denied	Access Denied	Pass

Table 5.2: System testing

5.1.5 Integration Testing

Integration testing assesses the interfaces and interactions between integrated components.

Test Case ID	Integration Component	Test Description	Expected Outcome	Actual Outcome	Status
IT-01	Preprocessing & Feature Extraction	Process X-ray and extract features	Valid feature vector	Valid feature vector	Pass
IT-02	Feature Extraction & Classification	Classify based on extracted features	Accurate diagnosis	Accurate diagnosis	Pass
IT-03	Genetic Algorithm & Classification	Optimize classification using GA	Improved accuracy	Improved accuracy	Pass

Table 5.3: Integration Testing

CHAPTER 6

FETURE ENHANCEMENT

6.1 Future Enhancements

While this project yielded promising results, there are several avenues for future enhancements:

1. **Incorporation of Additional Imaging Modalities**:
   - Future work could explore the integration of other imaging modalities, such as CT scans or MRI, alongside X-rays to provide a more comprehensive view of lung infections. Combining multiple imaging sources could improve diagnostic accuracy.
2. **Expansion of Dataset**:
   - Increasing the diversity and size of the dataset can enhance model robustness. Collaborating with healthcare institutions to gather more varied X-ray images of lung infections could provide additional training data.
3. **Real-Time Diagnosis**:
   - Implementing the developed model in a real-time diagnostic tool would allow healthcare professionals to receive immediate feedback during patient examinations. This could streamline the diagnostic process and improve patient management.
4. **Integration with Clinical Data**:
   - Combining radiomic features with clinical data, such as patient history and laboratory results, could further enhance the predictive power of the model. Developing a comprehensive decision support system that incorporates various data sources would be beneficial.
5. **Deep Learning Approaches**:
   - Exploring deep learning techniques, such as convolutional neural networks (CNNs), for feature extraction could provide even more powerful representations of X-ray images. This may lead to further improvements in diagnostic accuracy.

6. **User-Friendly Interface Development**:

- Creating a user-friendly interface for the model would facilitate its use by healthcare professionals. Ensuring ease of use is critical for adoption in clinical settings.
- By addressing these enhancements, future research can continue to improve the diagnostic capabilities for lung infections, ultimately leading to better patient outcomes and more effective healthcare solutions.

CHAPTER 7

SNAPSHOTS

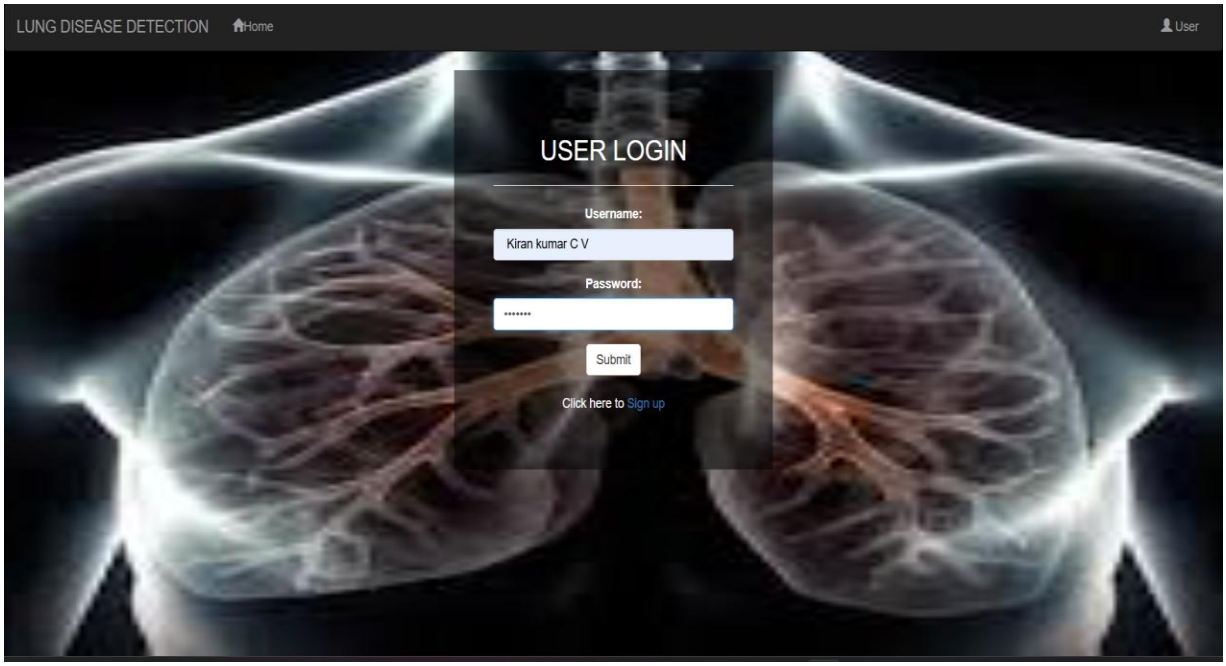


Fig 7.1: User login

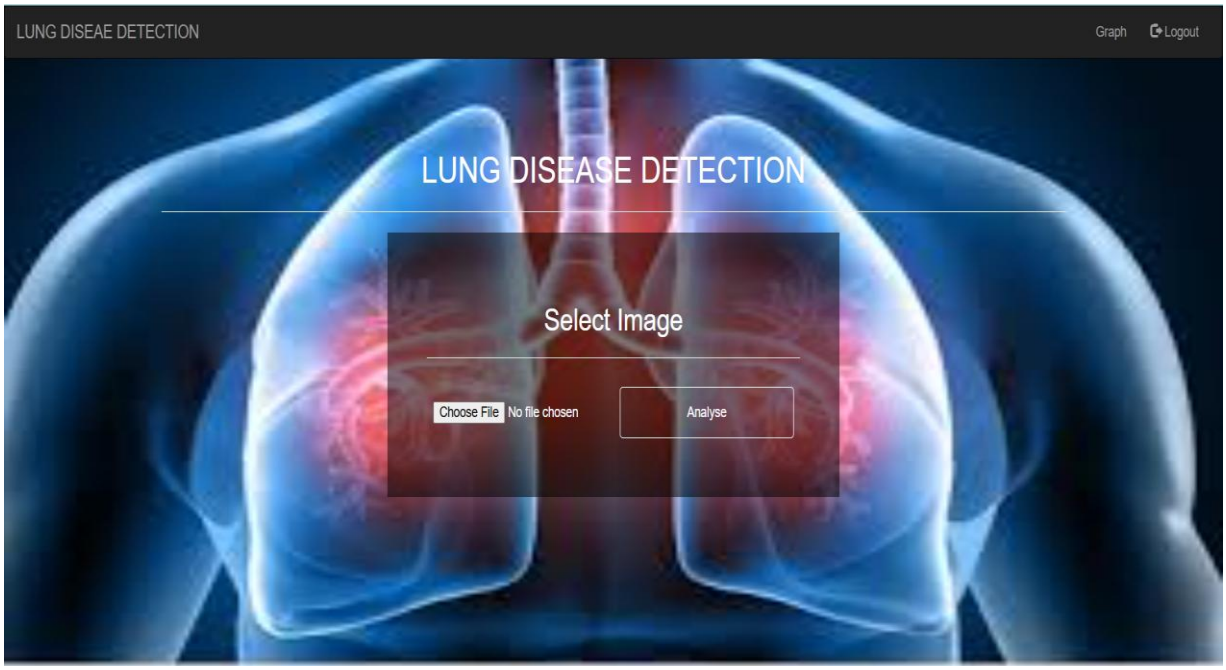


Fig 7.2: Home Page

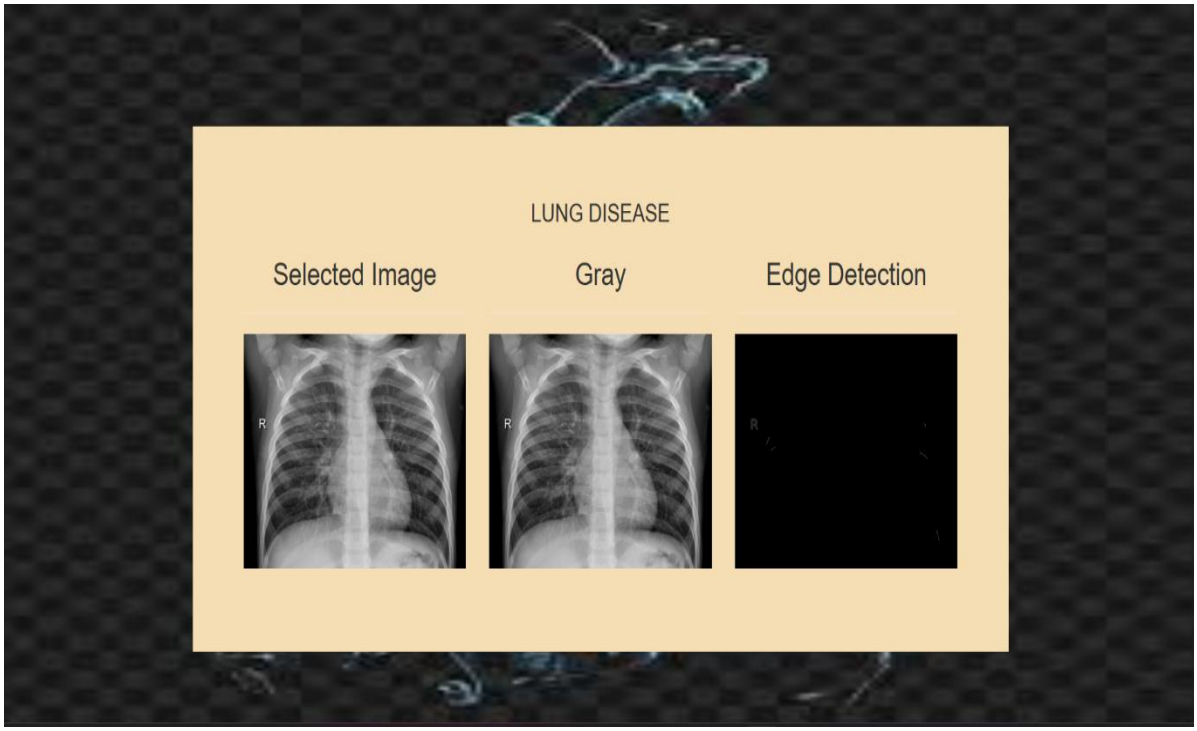


Fig 7.3: Image processing

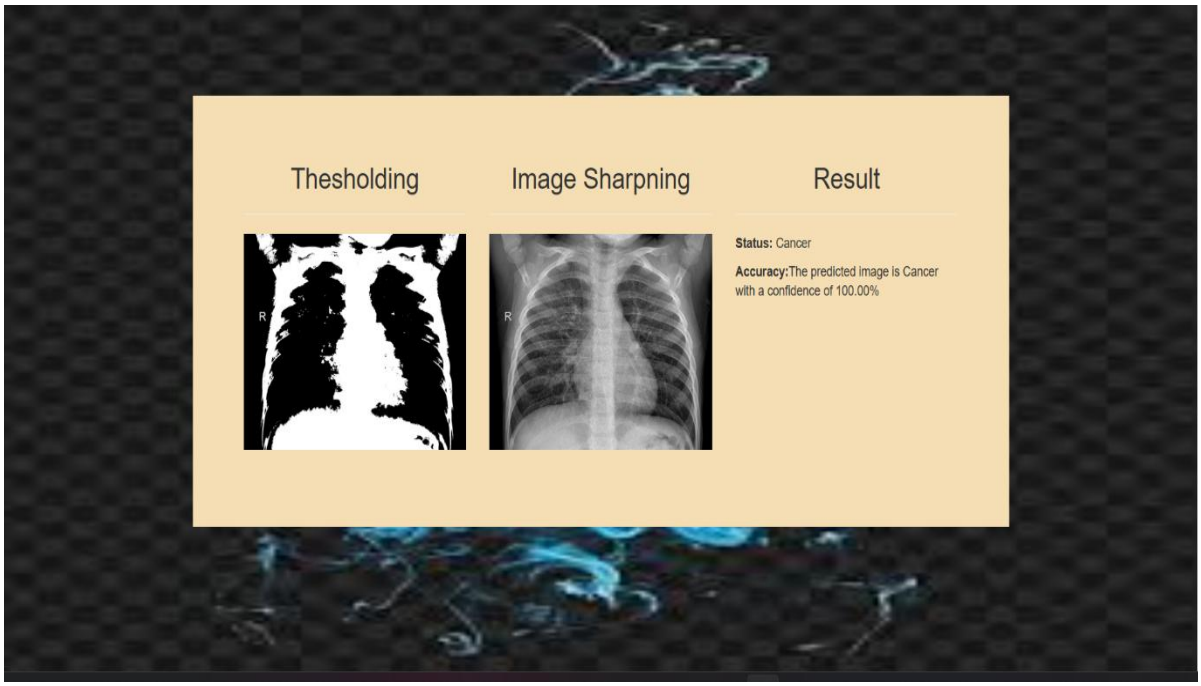


Fig 7.4: Result

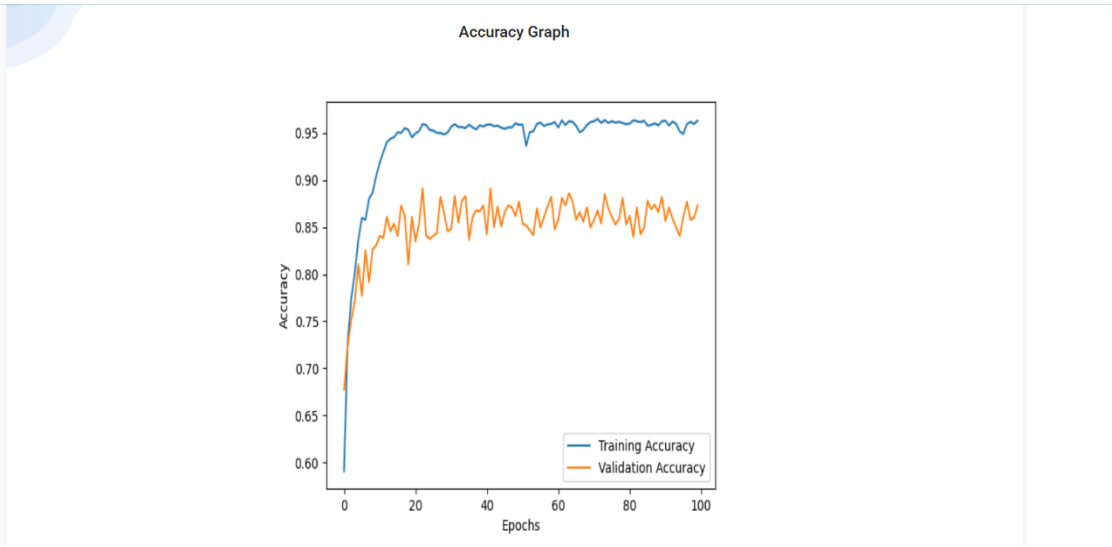


Fig 7.5: Accuracy Graph

CONCLUSION

The Arduino-based IoT Industry Protection System provides a cost-effective and scalable solution for enhancing safety and operational monitoring across various industrial sectors such as manufacturing plants, chemical processing units, and energy facilities. By leveraging integrated sensors and real-time monitoring capabilities, this system significantly improves hazard detection and risk management, helping prevent accidents caused by gas leaks, overheating, and environmental anomalies.

In the event of a system fault or emergency condition, immediate alerts are generated, allowing timely intervention to mitigate potential damage or hazards. Compared to traditional industrial safety systems, the Arduino-based IoT solution offers a more compact, affordable, and energy-efficient alternative, making it suitable for widespread deployment in both small- and large-scale operation.

REFERENCE

1. S. Sharma, R. Patel, and N. Gupta, “Lung Abnormality Identification With Explainable AI,” Int.J. Eng. Res. Technol., vol. 9, no. 7, pp. 354–358, July 2020. [Online]. Available: https://doi.org/10.17577/IJERTV9IS070267

2. Verma and D. Singh, “Smart Gas Detection System Using IoT,” Proc. Int. Conf. on Innovations in Computing Techniques (ICICT), Jaipur, India, 2019, pp. 21–25. [Online].Available: https://doi.org/10.1109/ICICT.2019.00011

3. R. Bansal, M. Das, and T. Roy, “Lung Abnormality Identification With Explainable AI,” Int. J. Sci. Res. Eng. Dev., vol. 4, no. 3, pp. 500–504, May 2021. [Online]. Available: https://doi.org/10.5281/zenodo.4770932

4. M. Kale, S. Yadav, and P. Joshi, “Lung Abnormality Identification With Explainable AI,” Int. J. Innov. Res. Comput. Commun. Eng., vol. 10, no. 2, pp. 1451–1457, Feb. 2022. [Online]. Available: https://doi.org/10.15680/IJIRCCE.2022.1002070